

Model Checking for Flexible Network Protocols

Andrew Johnson[1(✉)], Dennis Fetterly[2], Sean Song[2], and Jonathan Zolla[2]

[1] Princeton University, Princeton, NJ 08544, USA
aj3189@princeton.edu
[2] Google, Inc., Mountain View, CA 94043, USA
flexible-network-protocols-cav@google.com

Abstract. Operating a network is a daunting task. Operating one at a global scale, with stringent service objectives and requirements to be available during maintenance and failures, is even more so. At Google, we operate such a network. This paper details our experience applying formal methods to some of the networking protocols that are developed and maintained by in-house engineers. These protocols centrally route network traffic to respond to changes in demand, react to network failures, and allow for maintenance and upgrades.

We used formal methods to target a class of bugs stemming from unclear specifications, unintended system interactions, and logical errors at the specification level. We show how we modeled our protocols using an off-the-shelf model checker and a custom harness to scale the model horizontally. We were able to recreate several recent bugs and verify that the fixes implemented were correct. Finally, we present a method called state projection that we used to increase confidence in the coverage of our models, which we added to the TLC model checker for TLA+. We created 7 different TLA+ models and showed that they were effective at recreating bugs and verifying our fixes to those bugs.

1 Introduction

Google operates B4, one of the largest privately operated networks in the world [11,12], with stringent requirements for availability, bandwidth, and connectivity. It is managed with internally-developed control plane protocols for centralized traffic engineering, automatic failure recovery, and operator-initiated changes. A global scale and rapidly changing demand in response to traffic patterns or business needs stretch the network to operate in virtually every conceivable edge case. Furthermore, they are frequently updated. Engineers are able to fix bugs, add features, and rework the protocols with a release time of weeks to months; in contrast, changes to public internet protocols require coordination between many stakeholders, taking years and requiring backwards-compatibility.

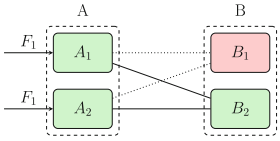
Control-plane protocols for B4 work at the abstraction level of *sites*, which are composed of control *domains* that contain physical network devices. Traffic

A. Johnson—Work completed while at Google.

© The Author(s) 2026
E. Darulova et al. (Eds.): CAV 2026, LNCS 16682, pp. 136–150, 2026.
https://doi.org/10.1007/978-3-032-32519-8_7

is grouped into *flows*, which transit the network via *tunnels* that consist of a list of sites. Traffic Engineering (TE) is a centralized system that periodically publishes a Traffic Engineering Database (TED), which is a plan for tunneling traffic in response to changing demand and network conditions. A TED entry is globally unique, and its value defines a mapping for a flow to a tunnel or a tunnel to sites. The goal of TE is to keep each domain's TED up to date with the centrally planned one. TED entries are added, removed, or modified on domains via *TE Ops*. Finally, each domain can be *OK* or *FailOpen*, determining whether or not it will accept TED changes. The global controller deals with failed domains by maintaining a global status (*Ready* or *NotReady*) for each domain. The pair of state and global status is a domain's *status*.

Figure 1a shows a sample up-to-date network with two sites, A and B, each with two domains. Figure 1b shows the corresponding global plan (TED), with flow F_1 mapped to tunnel T_1, defined as a tunnel from site A to B. Domain B_1 is in state $FailOpen$, shown with its red shading. B_1 is also globally $NotReady$, as shown by the dotted lines depicting traffic from site A to it.



(a) Topology

Key	Value
F_1	$\rightarrow T_1$
T_1	$\rightarrow A, B$

(b) Example global TED.

Fig. 1. A sample topology and corresponding TED. (Color figure online)

We modeled the installation of a TED on individual domains using TLA+ (Temporal Logic of Actions), a formal specification language [14,25]. TLA+ can be used to test *safety* properties, e.g. "Traffic never blackholes" or *liveness* properties, e.g. "TE Ops are eventually successful." A TLA+ model has constants, variables (the program state), an initial state formula, and a transition relation. TLA+ specifications can be used as mathematically precise documentation and as input to TLC [14], which is an explicit-state model checker for TLA+ models.

TLC was not able to handle our use case immediately. Including enough detail from our protocols to reproduce simple bugs became infeasible. The tool does not scale well horizontally because it must maintain a global set of previously seen states. Instead, we developed a way to split the protocol into distinct phases and horizontally parallelize our model by input (network topology and desired TED). This allowed us to run many instances of TLC which, when combined, explore the entire state space of the protocol. It also allowed us to perform symmetry reduction on the network topology, saving redundant work [5]. Finally, our production topology is too large for any model checker. Instead, we designed abstract interpretation-based summaries of the safety-critical portions of a state called state projections which provide a coverage criterion for exploration.

2 Model Description

The scope of our model is TED installation: TE Op generation, dependency computation, and installation order, as well as domain failure and recovery. We do not model the desired TED computation [11], instead assuming an optimal TED is given as input. We also do not model *link* failure and recovery, and exclude any transient network changes.

Constants. The specifics of the network topology are constants. This includes the sets of sites and domains, as well as the relationships between them.

Variables. In TLA+, variables are the primary drivers of exploration time, as they define the criteria for a unique state. Our model includes these variables:

1. Desired TED: The current desired TED
2. Domain TEDs: Map from domains to their current TED
3. Domain Status: Map from domains to their status
4. CurrOps: Ops that have yet to be complete
5. OpDependencies: Pairs of ops indicating a partial order of dependencies

Actions. Actions define the transition relation for a model. Ours has the following:

1. *ComputeOps*: Computes CurrOps (the difference between each Domain's initial TED and the desired TED) and OpDependencies. The calculation of the OpDependencies caused many of our targeted historical bugs. Example dependencies include requiring tunnels to be installed on all domains before flows that use them, or conversely removing a flow that uses a tunnel from all domains before removing the tunnel. Dependency computation also takes into account whether domains are *FailOpen* or *NotReady*.
2. *InstallOp*: The model can install any Op from CurrOps which does not depend on an incomplete Op and for which the domain is *OK* (a failed domain would result in a failed Op). We model all possible Op interleavings.
3. *DomainStatusChange*: Domains can fail or recover at any time. Many bugs arose from dependencies on failed domains. When a domain fails, it immediately changes its status to *FailOpen*. Then, it creates an Op (and updates dependencies) to change its status to *NotReady*, since this must be communicated from the central controller to neighboring domains. Similarly, a domain returning to *OK* status creates an Op to mark it *Ready*.
4. *ComputeTED*: At the end of each round, compute a new TED to install.

Once there are no more ops, the round is complete and we nondeterministically choose a new desired TED for the next round. Figure 2 shows a sample of one round of the protocol on the topology and desired TED from Fig. 1. The current TED is depicted by which lines are drawn. The initial state, shown in Fig. 2(c), begins with the current TED being empty in A_1 and B_1, and already matching the desired TED in A_2 and B_2, as shown by A_2 having an

arrow for F_1 while A_1 does not. Domains A_1, A_2, and B_2 begin with status *OK*, shown with the color green, while B_1 begins as *FailOpen*, shown with the color red. The dashed line from A_2 to B_1 indicates no traffic can flow on that link, since B_1 is *NotReady*, while the other domains are *Ready*. In Fig. 2(d), no change has occurred in the network, but the ops to install T_1 on A_1 and B_2 and F_1 on A_1 have been generated. Since F_1 uses T_1, it depends on both tunnel install ops. In Fig. 2(e), T_1 has been installed on A_1, and B_1 returns from *FailOpen* to *OK*. This creates a new op to mark it *Ready*, which depends on the tunnel install op. Figure 2(f) is the final state of the network after ops are completed.

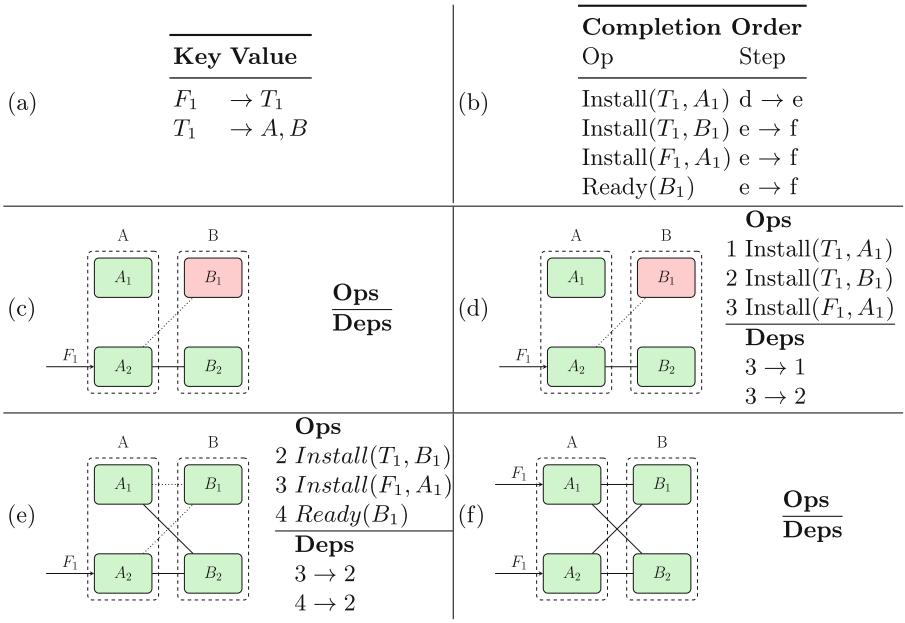


Fig. 2. This shows a sample run of our model with one domain recovering and a total of four ops. Part (a) shows the desired TED. Part (b) shows the order the ops are completed, and the steps between which they finish. Parts (c)–(f) show snapshots of the state at different points in the model run.

Invariants. We include the following definitions to explain our invariants:

Definition 1. *A tunnel t is* used *at domain d if there exists a domain d', flow f, and tunnel t such that at d', f maps to t and t includes the site where d is located.*

Definition 2. *A domain d is* blackholing *if there exists a tunnel t that is used at d, t is not installed at d, and d is ready. A state is* blackholing *if it contains a blackholing domain.*

The simplest invariant we check is that there are no blackholing domains (NoBlackHoles). We also wished to check the model in situations where the initial state may already contain blackholing traffic, which can be caused by factors external to the model. To do so, we utilized TLA+'s temporal faculties to specify a temporal version of the above property: Once a domain is not blackholing, it never becomes blackholing. This allows for us to start in blackholing states, but does not allow our logic to *cause* black holes.

3 Modeling Experience

Our process was to choose a past bug and create a TLA+ model that was as simple as possible while exhibiting the bug. Then, we implemented the bug's fix in the model and verified that the bug no longer appeared. We wished to model our protocols at a high enough fidelity to catch bugs and reflect the actual structure of our code. The further abstracted our models are from the implementation, the more likely mistakes become when translating any protocol changes into the model, causing spurious errors or missing edge cases.

We quickly ran into the scaling problem inherent to explicit-state model checking. B4 has hundreds of domains and thousands of flows and tunnels. At the fidelity we desired, model checking any topology with more than a few nodes and tunnels was infeasible. In this section, we describe how we built our model to take advantage of compute to scale short (single-round) runs out horizontally while giving us confidence for long (multi-round) runs.

3.1 Single-Round Model

The example from Sect. 2 describes a protocol with many rounds of TE updates, which we call the Tunnel Install (TI) Linear model. A round begins when a new desired TED is decided and ends when all of the TE Ops have either completed or failed. Once we added any more complexity, this became infeasible because it was not only testing op interleavings *within* rounds, but also all possible interleavings of *desired TEDs*. Instead, we model a single round, and then check every possible single-round run of the model, which we call the TI model. The TI and TI Linear models are equivalent in the states they explore, as we show in Sect. 5.

Instead of generating a new TED at the end of each round, the TI model changes the desired TED from a variable to a constant. We also add a new constant, which determines all the possible *starting* TEDs. This simulates all possible states the multi-round model could be in when the *ComputeTED* action ends the round. Each TED becomes a distinct starting state so that we do not miss states that would arise from rounds ending with incomplete ops.

To scale horizontally, we wrote a test harness in Python that enumerates all possible desired TEDs to use as input constants. We performed symmetry reduction on most topology elements at this stage to reduce duplicate work. For each desired TED generated after performing symmetry reduction, we run the model from every possible starting configuration of domain TEDs. The result is

running every starting configuration to every (symmetry-reduced) ending configuration, which is equivalent to the multi-round model. The key advantage of this is that each desired TED can be run as a separate instance of TLC. This allows us to easily parallelize them and scale model checking horizontally.

4 Projected States

It is impossible to run our model with a topology at the size of our production network. To increase confidence that our models have enough coverage on small topologies, we compute succinct, topology-agnostic abstractions of concrete states, which we call state *projections*. Intuitively, small topologies are "big enough" because, in our protocols, tunnels and flows are independent of one another and tunnel length does not matter. We are completing an exhaustive state space exploration, so we will explore every possible state that a short tunnel can reach, and the states of a long tunnel will be variations on that theme.

TLC uses hash fingerprints to keep track of previously seen states. We modified TLC to include another fingerprint for the state projection. Projections should be topology-agnostic; that is, the set of *possible* projections should be the same no matter the topology. Formally, a TLA+ model has a (possibly infinite) *concrete* state space $\mathcal{S}$ and a finite projected state space $\mathcal{A}$. The state projection, α, has domain $\mathcal{S}$ and range $\mathcal{P}(\mathcal{A})$ (the set of subsets of $\mathcal{A}$). This is most similar to classical predicate abstraction [2,6,9]. However, each concrete state maps to a set of per-domain summaries, making projections simpler to express. In our example model from Sect. 2, we define a projection as follows:

Definition 3. *A* domain projection *of a domain d and tunnel t is a tuple containing the following:*

1. *The domain's state, either OK or $FailOpen$*
2. *The domain's global status, either $Ready$ or $NotReady$.*
3. *A boolean indicating whether or not t is used at domain d.*
4. *A boolean indicating whether or not t is installed at d.*

The projection of a concrete state s is the set of domain projections for any combination of domain and tunnel.

This projection summarizes the safety-relevant state of each domain. The domain projection is enough to tell whether a domain is blackholing: a domain must be ready and used by a tunnel that is not installed. It is topology-agnostic as it does not scale with the number of domains or tunnels (there are 16 possible projections). This projection is hand-crafted, but is relatively clear from the system design. We verify in Sect. 6 that our models visit a state with every safe domain projection, and use this as a proxy for the coverage of our models. This does not provide a proof that no additional projected states can be discovered after a plateau, called stuttering. To provide such a proof, one would need to create an abstract program that operates over sets of projected states, or find another way to rule out stuttering behavior, such as with resource-unbounded analysis [13,15,16].

Implementation. We added a new configuration keyword, `PROJECTION`, to the TLC model checker that takes the name of a definition that is used to compute the projected state set, i.e. `PROJECTION proj_state_set`. The argument can be any TLA+ enumerable, meaning it can be expressed with the built-in TLA+ language features, including formulas containing state variables. The projection is evaluated in each state and enumerated, and each element is fingerprinted in a second fingerprint set. We compare fingerprint hashes directly across single-round runs by normalizing the value and using a fixed hash seed.

5 Evaluation

In evaluating our approach, we asked the following questions:

1. Did our models scale enough to give us confidence we are not missing bugs?
2. Were we able to complete model checking in a reasonable amount of time?
3. Did our single-round optimization of the tunnel installation protocol explore all the same states as the linear one?

5.1 Setup

We created the following models for testing:

1. Tunnel Install (TI) Linear: The model described in Sect. 2.
2. TI: The model described in Sect. 3.1.
3. TI NoFail: The same as TI, but without domain failure or recovery.
4. TI Groups (TI+G): This model adds the concept of Tunnel Groups to TI. Groups spread large flows across multiple tunnels, which may take different paths. This was required to recreate a bug which allowed tunnels to be added to groups before they were fully installed.
5. TI+G NoFail: The same as TI+G, but without domain failure or recovery.
6. Domain Drain: This is the network protocol used by the control plane to "drain" a domain, meaning moving traffic away from it. It is modeled at the same abstraction level as the Tunnel Install models, and is the implementation of the step in those models that changes a domain to or from "Ready". We used a projection that tracks a domain's status, readiness status, and the status of its connected external links.
7. Local Drain: This is the protocol used to drain a single network device *within* a domain. It is modeled at the level of a single domain. We used a projection that tracks a domain's status and the status of its connected external links.

We chose 5 sample topologies that all of the models were able to finish exploring, starting from the simplest possible and increasing in complexity. For example, with the TI models, topology 1 has two nodes and one tunnel and flow, while topology 5 has six nodes with two tunnels and two flows. They are equivalent up to differences required for the different models (e.g. the addition of tunnel groups). For each model, we wrote a Python test harness to determine all possible desired TEDs and initial states for a topology, up to symmetry reduction.

We only included desired TEDs that are *consistent*, meaning non-blackholing. We generate configurations in less than a second for all our examples. In order to properly compare the TI Linear (which never enters inconsistent states) with the single-round models, we only included non-blackholing starting states. Including blackholing starting states had similar performance and caused our models to reach all projected states. Each model is a correct version of the protocol after the bug fix was implemented. These are the most time consuming as they require TLC to run to completion; models of buggy versions discovered error traces quickly.

We ran each example TED as a separate run of TLC in its own job using Google's cluster management engine Borg [24,26] with 64GB memory. Our results are recorded in Table 1. The models are listed in column 1. In column 2, the topologies are numbered 1-5 in order of size. In column 3, we report how many different desired TEDs we ran after symmetry reduction (the number of tasks). In column 4, we list the total number of states generated from all runs, which is a good measure of total effort. In column 5, we list the number of projected states encountered across all tasks. In column 6, we list the average time runs took, while column 7 contains the time of the longest-running TED.

5.2 Results

We found that our models were able to scale past when the topology is big enough to saturate the (safe) projected state space. The two projected states left unexplored by all of the TI models are unsafe. For the models without domain failure, they never explore projected states with domain statuses other than (*OK, Ready*). For ones with domain failure, they explore these early and then plateau, exploring the same projected states with much larger topologies.

We also found that with the single-round model checking approach, our models all complete in a reasonable amount of time. The largest topology with the most complex model (TI+G with failure) takes less than an hour. Most complete in less than a minute. This is fast enough to be run repeatedly while updating the model. We had outlier topologies for the single-round approach (the max time was much greater than the average) due to combinatorial explosion in the initial state space for *empty* desired TEDs. We also created a linear model with groups, but it became infeasible for all but the smallest topology, which is evidence that our approach scales better than the linear one.

Finally, to verify that our single-round optimization does not miss states, we ran the TI model with no initial symmetry reduction alongside the TI Linear model. We had TLC dump the states for both and combined all of the dumped states from the single-round runs. Then, we verified that the single-round version explored the exact same states as the linear one by comparing the values of the variables in each state (excluding the additional variables required to track state across rounds). This was true for all of the TI Linear runs.

Table 1. We present our results for each example on 5 sample topologies.

Example	Topology	TEDs	States	Proj.States	Avg.Time(s)	MaxTime(s)
TI Linear	1	1	33	3	-	3
	2	1	6847	14	-	3
	3	1	388752	14	-	10
	4	1	3019565	14	-	13
	5	1	1118743386	14	-	2499
TI	1	3	48	3	0	0
	2	3	122603	14	1	1
	3	5	2382764	14	4	6
	4	8	5699172	14	5	7
	5	51	2607175615	14	240	2183
TI NoFail	1	3	48	3	0	0
	2	3	124	3	0	1
	3	5	699	3	1	1
	4	8	11230	3	1	1
	5	51	409669	3	2	4
TI+G	1	4	74	3	1	1
	2	4	64662	14	2	2
	3	7	4793197	14	4	7
	4	21	40445111	14	9	20
	5	105	29096754964	14	1003	3450
TI+G NoFail	1	4	74	3	1	1
	2	4	408	3	1	1
	3	7	1296	3	1	1
	4	21	225238	3	2	2
	5	105	3950608	3	2	5
Drain Domain	1	4	16	4	0	0
	2	6	411	12	0	0
	3	8	1763	12	1	1
	4	18	1275	12	1	1
	5	24	54156	12	1	1
Drain Local	1	1	13	4	0	0
	2	3	291	4	0	0
	3	12	6942	4	0	0
	4	12	184332	4	1	1
	5	60	184380	4	1	1

6 Discussion

Developer Effort. We found TLA+ to have a steep but short learning curve. Once overcome, each model took a developer about one week to create while learning the specifics of the design. Developing and testing multiple iterations of the single-round versions, complete with Python harnesses and test scripts for use in Google's Borg system took about two person-months, most of which produced reusable scaffolding. The "Drain" models (6 and 7) were created for future changes rather than previous bugs, and showed success with design-time modeling, as they helped find multiple errors in early protocol proposals and forced engineers to write their protocol change and desired properties precisely. This saved significant effort when compared to implementing and testing the changes in a network simulator, which may not have caught the errors. We also experimented with liveness properties such as verifying that all ops are eventually installed under different fairness assumptions, but focused on the NoBlackHoles safety property for the evaluation because it is equivalent between all of our examples. Finally, we found the single-round model useful in development because it was easy to run quickly (with limited starting states) while changing the protocol. It also made counterexamples easier to read than tracing a long, multi-round run.

Computational Cost. The primary challenge was ensuring our models ran efficiently. There is an inherent tension with models that are not provably connected to the runtime system: on one hand, we wished to optimize the models for faster runtimes and bigger topologies. On the other hand, the further the model gets from the real system, the more "hand-waving" is required to be sure one is not fabricating bugs, or worse, missing them. We needed a high level of abstraction in order to get something different from a network simulator for testing, which is already done extensively on all B4 changes.

Extensibility. We faced unique challenges in this project that depart from one-shot academic verification projects, including the need for fast changes, a global scale, and a long list of specific problems to target. We think two main takeaways are useful for developers wishing to apply similar methods.

First, the "single-round" approach was successful because it gave a start and end to a reactive system, which was possible because the single TE process acted as a leader that defined the rounds. We believe there are many problems in networking and other areas that have layers of abstraction where one can perform the "outer loop" in parallel to increase compute efficiency. For example, BGP could treat each route announcement as a "single round." In networking specifically, control plane protocols are a good match for model checking because they are insulated by well-defined abstraction layers from concerns like individual packets and physical links. It also allowed us to perform symmetry reduction at the example level with symmetries that could not be applied at the level of a fixed model. Since there was no single state set, the number of different configurations could be very large and we were able to run dozens of jobs in parallel.

Second, pen-and-paper design documents were a useful level of abstraction for formal modeling. They are early enough in the development process to catch bugs before incurring the cost of bringing a change to a testable state. Google employs a wide variety of network testing schemes, but all require a change to be designed and at least partially implemented, whereas model-checking is able to find errors at the pseudo-code level.

Limitations. We left as out of scope errors that occur between the network devices and domain controllers, as well as a host of other concerns such as the mechanics of retransmitted messages, hardware malfunctions other than total failures, and bugs in unmodeled systems. We also do not model every piece of the protocol, instead choosing to simplify models as much as possible while still recreating real-world bugs. A monolithic model would be infeasible for model checking, so we hope to have a range of simplified models (about one per design document) to allow engineers to choose the best abstraction level.

The single-round method improves performance, but still suffers from combinatorial explosion. Techniques for sharing the fingerprint state set across parallel computations, such as those developed for DiVinE [3], could aid in reducing repeated work. We found our approach to scale better than the distributed mode of TLC, which also cannot check liveness properties. Furthermore, for very large topologies, generating (and symmetry reducing) possible configurations could also suffer, though this might be avoided with better reduction techniques [7].

Currently, temporal formulas cannot be included in state projections. While it was not required for our models, this would be an interesting avenue for future work and would allow for more complex projections. We would also like to close the Abstract Interpretation loop for projected states by developing a way to compute an abstract program from a TLA+ model. This could be especially challenging if temporal formulas were added to projections, but would give a verification procedure for topologies of any size.

7 Related Work

There is a wide range of network verification systems that have been developed, each with different tradeoffs of user effort to guarantee strength. On one end of the spectrum are systems such as IronFleet, Verdi, Ivy, I4, Minesweeper, CCAC, and TimePiece, which all focus on finding strong inductive invariants of distributed systems [1,10,17,22,27]. Because they require significant dedicated user effort, they are meant for widely-used systems such as public-internet protocols or consensus protocols. A similar large, one-time effort to formalize and verify Google's algorithms would be difficult to maintain across protocol changes. Efforts to lower this cost such as DistAI, which attempts to automatically generate invariants, and Grove, which employs separation logic to modularize proofs, still require protocol specifications in a proof language [23,28]. A model-checking approach makes the most sense for our use case, as model complexity matches the level of design documentation and changes will be similar and proportional to implementation changes.

Symmetry reduction has been used in many forms to tackle state space explosion, for example in hardware verification [19], parameterized systems [4,8], and symbolic model checking [7,20]. These approaches provide strong theoretical guarantees but would require implementation in TLC. The best available tools are TLC's refinement checker or Apalache [21], which still require the user to provide the entire abstract program in addition to the abstraction. State projections are a practical starting point for defining abstractions without symbolic execution or BDD machinery.

In contrast to proof-based systems, network *runtime* verification systems such as Aragog or Pivot Tracing are automated but incur a runtime cost and only check events that they observe directly [18,29]. Google already uses runtime verification extensively. Our approach is in the middle: it can find bugs at the level of *design documents*, is easy to set up and modify as it does not touch the production system, and does not require modeling the entirety of the network.

8 Conclusion

We presented our experience using TLA+ to model some of the control plane protocols running on Google's B4. We modeled various protocols at a high level of abstraction akin to a design document. To perform model checking on them, we present a novel technique to horizontally parallelize model checking in our specific domain. Furthermore, we added an abstraction mechanism to TLA+ to ensure we explored all abstract states of interest to developers. We evaluated our techniques on various network topologies and showed that they made the model checking problem feasible.

Acknowledgments. We thank Subhasree Mandal, Aman Shaikh, and the anonymous CAV reviewers for their comments on earlier drafts of this paper. We also thank Nathan Flath, Kal Nasser, Li Qiu, Shuanglong Zhang, and Yang Zhang for their contributions throughout the project.

Data Availability Statement. The data and artifact for this paper were not made publicly available to retain intellectual property.

Disclosure of Interests. Andrew Johnson was employed as a Student Researcher at Google, Inc. while this work was completed. The authors have no competing interests.

References

1. Alberdingk Thijm, T., Beckett, R., Gupta, A., Walker, D.: Modular control plane verification via temporal invariants. Proc. ACM Program. Lang. **7**(PLDI), 50–75 (2023). ISSN: 2475-1421. https://doi.org/10.1145/3591222
2. Ball, T., Majumdar, R., Millstein, T., Rajamani, S.K.: Automatic predicate abstraction of C programs. SIGPLAN Not. **36**(5), 203–213 (2001). ISSN: 0362-1340. https://doi.org/10.1145/381694.378846

3. Barnat, J., et al.: DiVinE 3.0 – an explicit-state model checker for multithreaded C & C++ programs. In: Sharygina, N., Veith, H. (eds.) CAV 2013. LNCS, vol. 8044, pp. 863–868. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39799-8_60
4. Chou, C.-T., Mannava, P.K., Park, S.: A simple method for parameterized verification of cache coherence protocols. In: Hu, A.J., Martin, A.K. (eds.) FMCAD 2004. LNCS, vol. 3312, pp. 382–398. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30494-4_27
5. Clarke, E.M., Emerson, E.A., Jha, S., Sistla, A.P.: Symmetry reductions in model checking. In: Hu, A.J., Vardi, M.Y. (eds.) CAV 1998, pp. 147–158. Springer, Heidelberg (1998). ISBN: 978-3-540-69339-0
6. Cousot, P., Cousot, R.: Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, POPL 1977, Los Angeles, California, pp. 238–252. Association for Computing Machinery (1977). ISBN: 9781450373500. https://doi.org/10.1145/512950.512973
7. Emerson, E.A., Wahl, T.: Dynamic symmetry reduction. In: Halbwachs, N., Zuck, L.D. (eds.) TACAS 2005. LNCS, vol. 3440, pp. 382–396. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-31980-1_25
8. Fang, Y., McMillan, K.L., Pnueli, A., Zuck, L.D.: Liveness by invisible invariants. In: Najm, E., Pradat-Peyre, J.-F., Donzeau-Gouge, V.V. (eds.) FORTE 2006. LNCS, vol. 4229, pp. 356–371. Springer, Heidelberg (2006). https://doi.org/10.1007/11888116_26
9. Flanagan, C., Qadeer, S.: Predicate abstraction for software verification. SIGPLAN Not. **37**(1), 191–202 (2002). ISSN: 0362-1340. https://doi.org/10.1145/565816.503291
10. Hawblitzel, C., et al.: IronFleet: proving practical distributed systems correct. In: Proceedings of the 25th Symposium on Operating Systems Principles. SOSP 2015, Monterey, California, pp. 1–17. Association for Computing Machinery (2015). ISBN: 9781450338349. https://doi.org/10.1145/2815400.2815428
11. Hong, C.-Y., et al.: B4 and after: managing hierarchy, partitioning, and asymmetry for availability and scale in Google's software-defined WAN. In: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication. SIGCOMM 2018, Budapest, Hungary, pp. 74–87. Association for Computing Machinery, 2018. ISBN: 9781450355674. https://doi.org/10.1145/3230543.3230545
12. Jain, S., et al.: B4: experience with a globally-deployed software defined wan. SIGCOMM Comput. Commun. Rev. **43**(4), 3–14 (2013). ISSN: 0146-4833. https://doi.org/10.1145/2534169.2486019
13. Johnson, A., Wahl, T.: Delay-bounded scheduling without delay! In: Silva, A., Leino, K.R.M. (eds.) CAV 2021. LNCS, vol. 12759, pp. 380–402. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-81685-8_18
14. Lamport, L.: Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley Longman Publishing Co., Inc., USA (2002). ISBN: 032114306X
15. Liu, P., Wahl, T.: CUBA: interprocedural context-unbounded analysis of concurrent programs. In: Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation. PLDI 2018, Philadelphia, PA, USA, pp. 105–119. Association for Computing Machinery (2018). ISBN: 9781450356985. https://doi.org/10.1145/3192366.3192419

16. Liu, P., Wahl, T., Lal, A.: Verifying asynchronous event-driven programs using partial abstract transformers. In: Dillig, I., Tasiran, S. (eds.) CAV 2019. LNCS, vol. 11562, pp. 386–404. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-25543-5_22
17. Ma, H., Goel, A., Jeannin, J.B., Kapritsos, M., Kasikci, B., Sakallah, K.A.: I4: incremental inference of inductive invariants for verification of distributed protocols. In: Proceedings of the 27th ACM Symposium on Operating Systems Principles. SOSP 2019, Huntsville, Ontario, Canada, pp. 370–384. Association for Computing Machinery (2019). ISBN: 9781450368735. https://doi.org/10.1145/3341301.3359651
18. Mace, J., Roelke, R., Fonseca, R.: Pivot tracing: dynamic causal monitoring for distributed systems. In: Proceedings of the 25th Symposium on Operating Systems Principles. SOSP 2015, Monterey, California, pp. 378–393. Association for Computing Machinery (2015). ISBN: 9781450338349. https://doi.org/10.1145/2815400.2815415
19. McMillan, K.L.: A methodology for hardware verification using compositional model checking. Sci. Comput. Program. **37**(1), 279–309 (2000). ISSN: 0167-6423. https://doi.org/10.1016/S0167-6423(99)00030-1
20. McMillan, K.L.: Eager abstraction for symbolic model checking. In: Chockler, H., Weissenbacher, G. (eds.) CAV 2018. LNCS, vol. 10981, pp. 191–208. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-96145-3_11
21. Otoni, R., Konnov, I., Kukovec, J., Eugster, P., Sharygina, N.: Symbolic model checking for TLA+ made faster. In: Sankaranarayanan, S., Sharygina, N. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, pp. 126–144. Springer, Cham (2023). ISBN: 978-3-031-30823-9. https://doi.org/10.1007/978-3-031-30823-9_7
22. Padon, O., McMillan, K. L., Panda, A., Sagiv, M., Shoham, S.: Ivy: safety verification by interactive generalization. SIGPLAN Not. **51**(6), 614–630 (2016). ISSN: 0362-1340. https://doi.org/10.1145/2980983.2908118
23. Sharma, U., Jung, R., Tassarotti, J., Kaashoek, F., Zeldovich, N.: Grove: a separation-logic library for verifying distributed systems. In: Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, pp. 113–129. Association for Computing Machinery (2023). ISBN: 9798400702297. https://doi.org/10.1145/3600006.3613172
24. Tirmazi, M., et al.: Borg: the next generation. In: Proceedings of the Fifteenth European Conference on Computer Systems. EuroSys 2020, Heraklion, Greece. Association for Computing Machinery (2020). ISBN: 9781450368827. https://doi.org/10.1145/3342195.3387517
25. tlaplus. TLA+ Model Checker and Toolbox Source Code (2025). https://github.com/tlaplus/tlaplus. GitHub repository. Accessed 8 Oct 2025
26. Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., Wilkes, J.: Large-scale cluster management at Google with Borg. In: Proceedings of the Tenth European Conference on Computer Systems. EuroSys 2015, Bordeaux, France. Association for Computing Machinery (2015). ISBN: 9781450332385. https://doi.org/10.1145/2741948.2741964
27. Wilcox, J.R., et al.: Verdi: a framework for implementing and formally verifying distributed systems. SIGPLAN Not. **50**(6), 357–368 (2015). ISSN: 0362-1340. https://doi.org/10.1145/2813885.2737958

28. Yao, J., Tao, R., Gu, R., Nieh, J., Jana, S., Ryan, G.: DistAI: data-driven automated invariant learning for distributed protocols. In: 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2021), pp. 405–421. USENIX Association (2021). ISBN: 978-1-939133-22-9
29. Yaseen, N., Arzani, B., Beckett, R., Ciraci, S., Liu, V.: Aragog: scalable runtime verification of shardable networked systems. In: Operating systems and implementations (OSDI) (2020). https://www.microsoft.com/en-us/research/publication/aragogscalable-runtime-verification-of-shardable-networked-systems/

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

